

Software Engineering

Fall 2026

LECTURE 3 - 4

Shahid Manzoor
shahid.manzoor@pucit.edu.pk

Part I

REQUIREMENTS ENGINEERING
TYPE OF REQUIREMENTS

Requirements Engineering

Requirements are about **WHAT**,
not HOW

DEFINITION: *Requirements are a specification of what should be implemented. They are descriptions of how the system should behave, or of a system property or attribute. They may be a constraint on the development process of the system.*

Key Slide!

Requirements Engineering.

Customers

- **Stakeholder** is a person, group, or organization that
 - is actively involved in a project, or
 - is affected by its process or outcome,
 - or can influence its process or outcome.
 - Stakeholders can be internal or external to the project team and to the developing organization.
- A **customer** is an individual or organization that derives either direct or indirect benefit from a product.
- **Users** (*aka end users*) are a subset of customers who will actually use the product, either directly or indirectly.
 - *Direct users* will operate the product hands-on.
 - *Indirect users* might receive outputs from the system without touching it themselves.

Requirements Engineering.

Customers

- **Stakeholder** is a person, group, or organization that
 - is actively involved in a project, or
 - is affected by its process or outcome,
 - or can influence its process or outcome.
- Stakeholders can be internal or external to the project team and to the developing organization.
- A **customer** is an individual or organization that derives either direct or indirect benefit from a product.
- **Users** (*aka end users*) are a subset of customers who will actually use the product, either directly or indirectly.
 - *Direct users* will operate the product hands-on.
 - *Indirect users* might receive outputs from the system without touching it themselves.

Requirements Engineering.

Customers

- **Stakeholder** is a person, group, or organization that
 - is actively involved in a project, or
 - is affected by its process or outcome,
 - or can influence its process or outcome.
- Stakeholders can be internal or external to the project team and to the developing organization.
- A **customer** is an individual or organization that derives either direct or indirect benefit from a product.
- **Users** (*aka end users*) are a subset of customers who will actually use the product, either directly or indirectly.
 - *Direct users* will operate the product hands-on.
 - *Indirect users* might receive outputs from the system without touching it themselves.

1. ELICITATION | ← Gather needs from stakeholders



2. ANALYSIS | ← Detect conflicts, gaps, ambiguity



3. SPECIFICATION | ← Document formally (SRS)



4. VALIDATION | ← Confirm you built the RIGHT thing



5. MANAGEMENT | ← Handle changes over time

Interviews	Deep understanding	Ask homeowner: "When do you want it silent?"
Questionnaires	Large user base	Survey 100 buyers on preferred chime volume
Brainstorming	New/innovative features	"What if it syncs with your phone?"
Observation	Tacit requirements	Watch how owner winds it daily
Document analysis	Existing systems	Read old clock's manual
Prototyping	Unclear requirements	Build cardboard mockup, get feedback
Focus groups	Consensus building	Gather 8 users to discuss features
Use cases	Functional flows	"User silences clock at 10 PM"

Conflict resolution	Resolve contradictory requirements
Prioritization	MoSCoW (Must/Should/Could/Won't), Kano model
Feasibility check	Can we build it within budget/time/tech?
Modeling	UML, state machines, data flow diagrams
Negotiation	Balance stakeholder needs
Risk analysis	Identify what could go wrong

The SRS (Software Requirements Specification)

The IEEE 830 standard defines the classic SRS structure:

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions
 - 1.4 References
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Characteristics
 - 2.4 Constraints
 - 2.5 Assumptions
- 3. Specific Requirements
 - 3.1 Functional Requirements
 - 3.2 Non-Functional Requirements
 - 3.3 External Interfaces
 - 3.4 Data Requirements
- 4. Appendices
- 5. Index

Characteristics of a Good Requirement

Characteristic	Bad Example	Good Example
Correct	"Clock chimes 5 times at 6:00"	"Chime count = current hour"
Unambiguous	"Fast response"	"Chime starts within 2 sec of HH:00"
Complete	"Chime on hour"	"Chime on hour AND half-hour"
Consistent	"Silent at night" + "Always chime"	Resolved: "Silent mode overrides chime"
Verifiable	"User-friendly"	"Time-setting in ≤ 3 steps"
Feasible	"Runs forever without winding"	"Runs 168 hours per winding"
Traceable	No ID	"SR-04 traces to UR-02"
Necessary	"Gold-plated feature"	Tied to a real user need

Requirement ID Scheme

- UR-XX → User Requirement
- FR-XX → Functional Requirement
- NFR-XX → Non-Functional Requirement
- SR-XX → System Requirement

Validation — "Did We Build the RIGHT Thing?"

Validation: "Are we building the right product?" (requirements correct?)

Verification: "Are we building the product right?" (code matches spec?)

Technique	Description
Reviews / walkthroughs	Stakeholders read and critique SRS
Prototyping	Show mockup, get feedback
Test case generation	If you can't write a test, requirement is unclear
Traceability matrix	Every requirement maps to a source and a test
Formal inspection	Structured defect detection

Management: "Handling Change Over Time"

Control changes so the project doesn't spiral.

Practice	Purpose
Baseline	Freeze an agreed version of requirements
Change control board (CCB)	Approve/reject change requests
Impact analysis	Assess cost/schedule/risk of each change
Traceability matrix	Link requirements → design → code → tests
Version control	Track requirement evolution
Scope management	Prevent scope creep

The Traceability Matrix

UR	FR	Design	Test	Status
UR-01 (correct time)	FR-01	Pendulum module	T-01	Pass
UR-02 (hourly chime)	FR-03, FR-04	Chime controller	T-03	Pass
UR-03 (silence)	FR-06	Silence switch	T-06	In progress

User vs. System Requirements

Q: What is a cuckoo clock?

User vs. System Requirements



Example 1: Timekeeping

- User Requirement
- "The user wants to see the correct time."
- System Requirement

"The system shall maintain time with a drift of $\leq \pm 1$ second per day using a pendulum or quartz oscillator."

Example 2: Chime

- User Requirement

"The user wants to hear the cuckoo at the right hour."

System Requirement

"The system shall trigger N cuckoo cycles where N = current hour (1–12), at HH:00 ± 2 seconds, producing a two-tone call at 800Hz and 600Hz."

Example 3: Silence

- User Requirement

"The user wants to mute the clock at night."

- System Requirement

"The system shall accept a binary silence input that suppresses all acoustic output while preserving timekeeping and chime-count state."

Example 4: Reliability

- User Requirement

- "The user expects the clock to last for years."

System Requirement

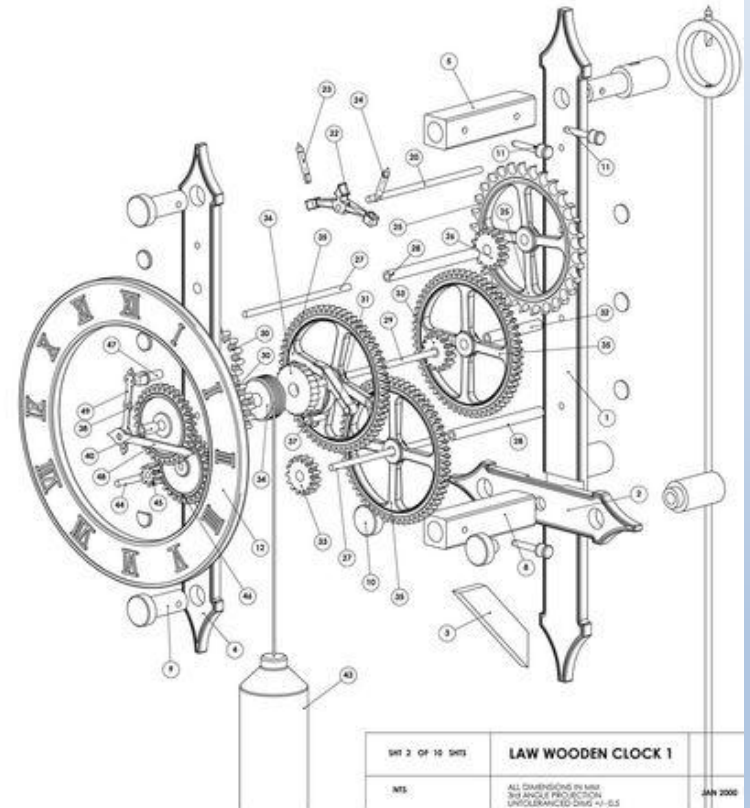
- "The system shall achieve an MTBF of ≥ 5 years under operating conditions of 10–30°C and 30–70% humidity."

Aspect	User Requirement	System Requirement
Audience	Customer, end user, stakeholder	Developers, testers, architects
Language	Everyday language	Technical, formal, precise
Focus	Problem space	Solution space
Measurable?	Often qualitative	Always quantitative
Testable?	Indirectly (via system reqs)	Directly (unit/integration tests)
Stability	Changes when business needs change	Changes when design/tech changes
Granularity	Coarse (one per need)	Fine (many per user req)
Example verb	"wants," "needs," "expects"	"shall," "must," "should"
Document	Vision doc, user stories, backlog	SRS, design spec, API contract
Who writes it	Product owner / BA	System engineer / architect

User vs. System Requirements



ITEM NO.	QTY.	PART NO.	DESCRIPTION
1	1	Back	
2	1	Cross	
3	1	Angled brace	
4	1	Front	
5	1	Squirrelnose top	
6	1	Pendulum hanger	
7	2	Wall spacer	
8	1	Squirrelnose bottom	
9	2	Front pin	
10	2	Covert pin	
11	3	Securing pin	
12	1	Clock dial	
13	2	Dial spacer	
14	1	Pendulum head	
15	1	Pendulum point	
16	1	Pendulum rod	
17	1	Pendulum bob	
18	1	Pendulum nut	
19	6	Shaft cover	
20	1	Shaft 151	
21	1	Yoke	
22	1	Escape	
23	1	Pinlet 2	
24	1	Pinlet 1	
25	1	Timing	
26	1	16teeth	
27	2	Shaft 140	
28	2	Sleeve 119	
29	1	Shaft 175	
30	2	15teeth 1.5	
31	1	Pinlet	
32	1	Sleeve 70	
33	2	15teeth	
34	1	Drum	
35	3	60teeth	
36	1	Ratchet	
37	1	Pinlet pin	
38	1	30teeth	
39	1	15teeth	
40	1	Sleeve 18	
41	8	Risping	
42	1	Rope	
43	1	Weight	
44	1	Shaft 130	
45	1	8teeth	
46	1	30teeth	
47	1	Key shaft	
48	1	Minute hand	
49	1	Hour hand	



User vs. System Requirements

1. User Requirements

- Goal or task that **specific classes of users must** be able to perform with a system, or
- A **desired** product attribute

2. System Requirements

- Top-level requirement for a product
- Software or software + hardware

Key Slide!

User vs. System Requirements

- **User**

- Abstract
- Natural Language
- What customers want

- **System**

- More concrete, detailed
- Natural + Formal language
- What system provides
- Used as product description

A Restaurant example

- **User**

- Food should look good
- Food should taste good
- Diners want enough food
- Diners want fish
- Chefs want challenging dishes

- **System**

- Chicken soup
- Baked salmon with ...
- Chocolate mousse

A Software example

- **User Requirements**

- *I need to print a mailing label for a package*
- *As the lead machine operator, I need to calibrate the pump controller first thing every morning*

- **System Requirements**

- If the pressure exceeds 40.0 psi, the high-pressure warning light should come on.
- The user must be able to sort the project list in forward and reverse alphabetical order.

A Software example

User requirements definition

- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

1. User Requirements

- Written as use cases / user stories (*agile*)
- Informed by **0. Domain Requirements**: “*everyone knows that...*”
- **Use Case**
 - Sequence of interactions between a system and external actor to achieve an outcome of value
- **User Story**
 - Short, simple description of a feature told from the perspective of who wants it (e.g., user or customer) from the system
 - As a **USER ROLE**, I want **DO SOMETHING** so that **REASON**

1. User Requirements

- Written as use cases / user stories (*agile*)
- Informed by **0. Domain Requirements**: “*everyone knows that...*”

TABLE 8-2 Some sample use cases and corresponding user stories

Application	Sample use case	Corresponding user story
Chemical tracking system	Request a Chemical	As a chemist, I want to request a chemical so that I can perform experiments.
Airport check-in kiosk	Check in for a Flight	As a traveler, I want to check in for a flight so that I can fly to my destination.
Accounting system	Create an Invoice	As a small business owner, I want to create an invoice so that I can bill a customer.
Online bookstore	Update Customer Profile	As a customer, I want to update my customer profile so that future purchases are billed to a new credit card number.

who wants it (e.g., user or customer) from the system

– As a USER ROLE, I want DO SOMETHING so that REASON

1. User Requirements

- **User classes**
 - Subset of product's customers
 - An individual can belong to several classes
 - Each user class must have a set of requirements
 - Classes need not be human beings
- **What process to use?**
 - User personas
 - A description of a representative of a user class with similar characteristics and needs
 - User representatives
 - A suitable representative to provide the voice of the user
 - Product Champion
 - An intermediary gathering requirements from the user for us
- In agile methods, all user classes are represented by the **Product Owner**

2. System Requirements

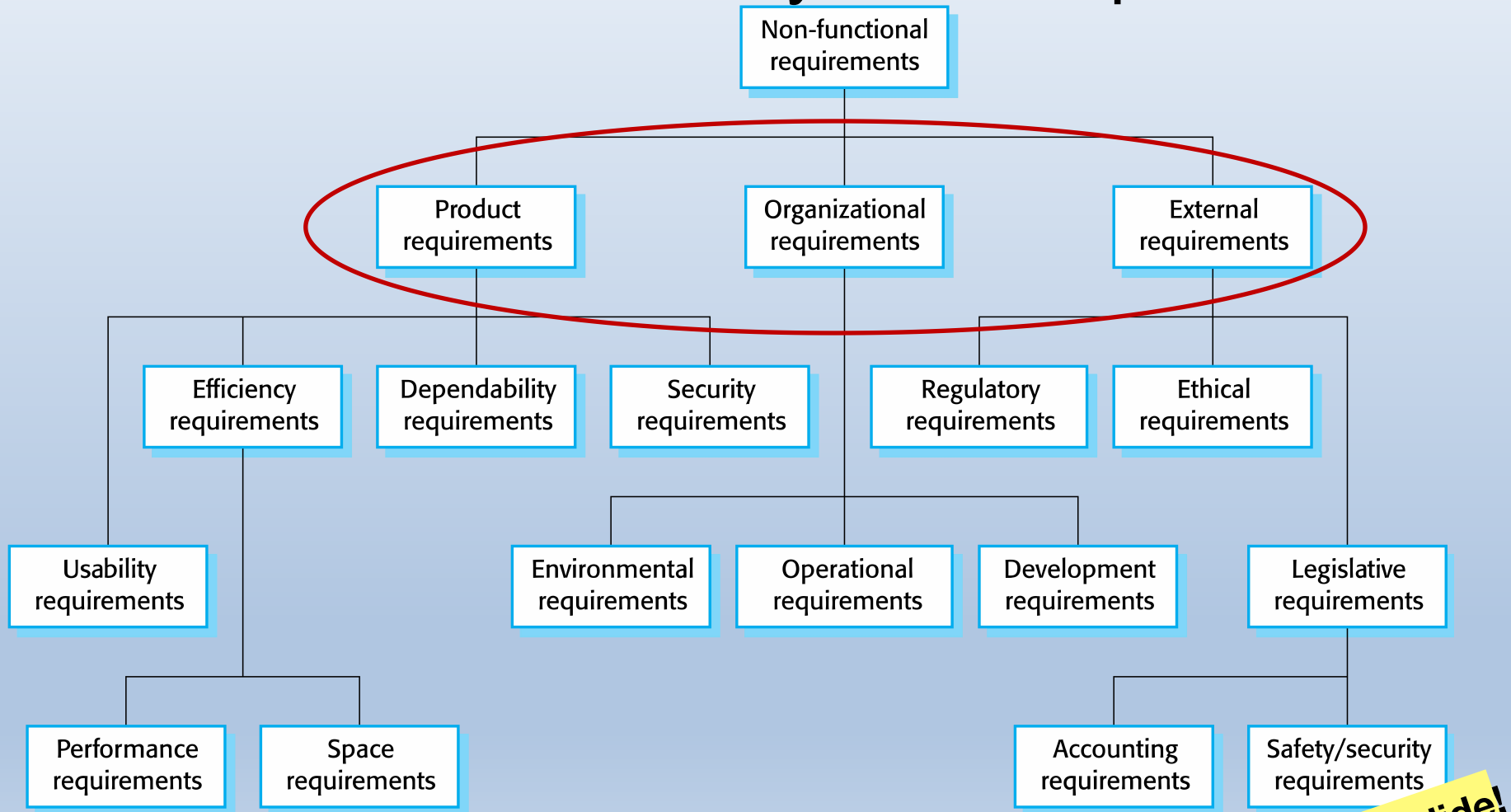
- Divided in two categories
- Informed by **0. Domain Requirements:** *“everyone knows that...”*
- **2.1. Functional Requirements**
 - Behavior that a system will exhibit under specific conditions
 - Describes *what* developers must implement to satisfy user requirements
 - Written as “*shall*” statements
- **2.2. Non-functional Requirements**
 - Describe *how well* a system does what it must do
 - Quality standard of the system
 - Must be measurable

Key Slide!

2.1. Functional System Requirements

- Behavior that a system will exhibit under specific conditions
- **Examples**
 - *The Passenger **shall be able to** print boarding passes for all flight segments for which he has checked in.*
 - *If the Passenger's profile does not indicate a seating preference, the reservation system **shall** assign a seat.*

2.2. Non-Functional System Requirements



Key Slide!

Metrics for Non-functional System Requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Questions

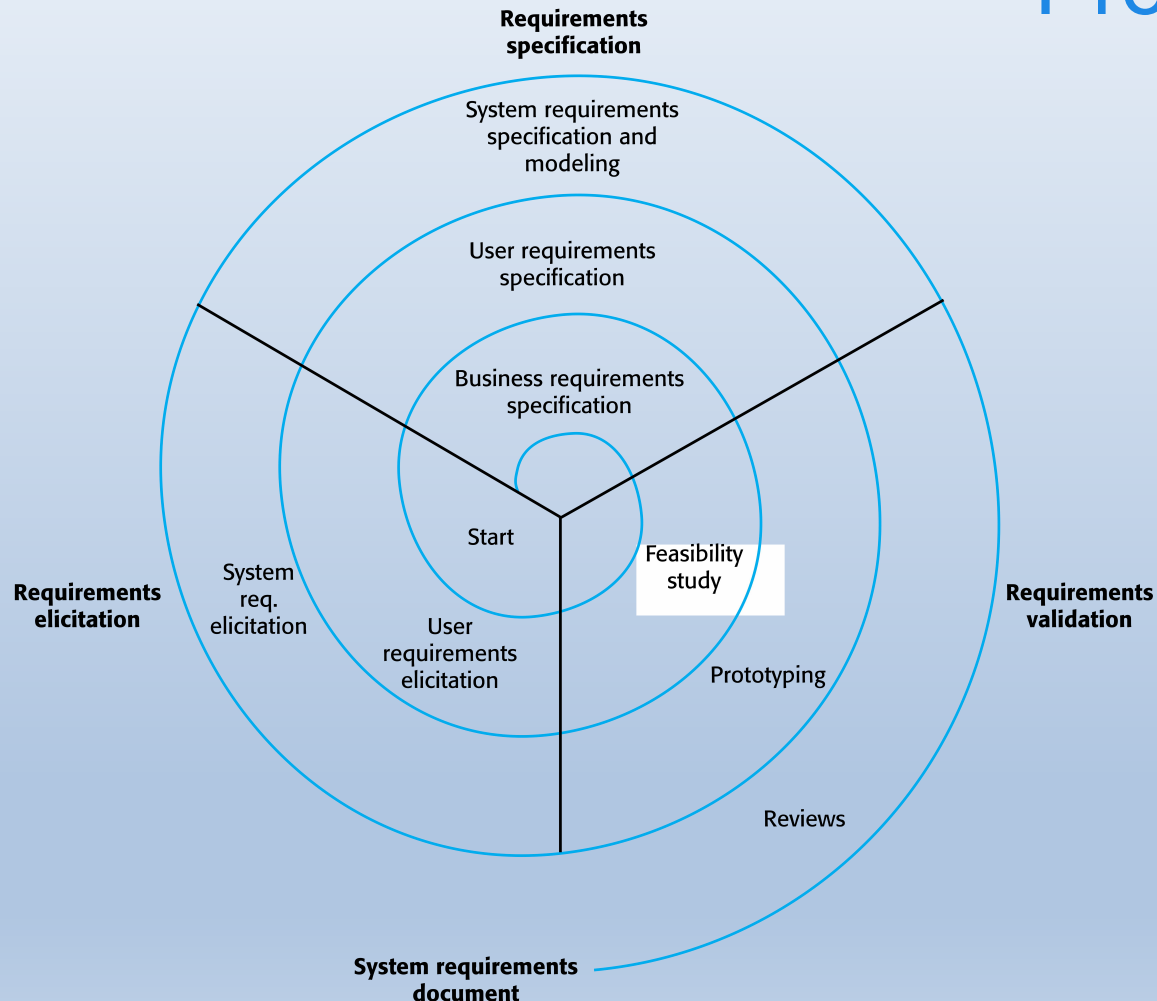


Part II

REQUIREMENTS ENGINEERING **PROCESS**

Requirements Engineering

Process I



Requirements Engineering Process II

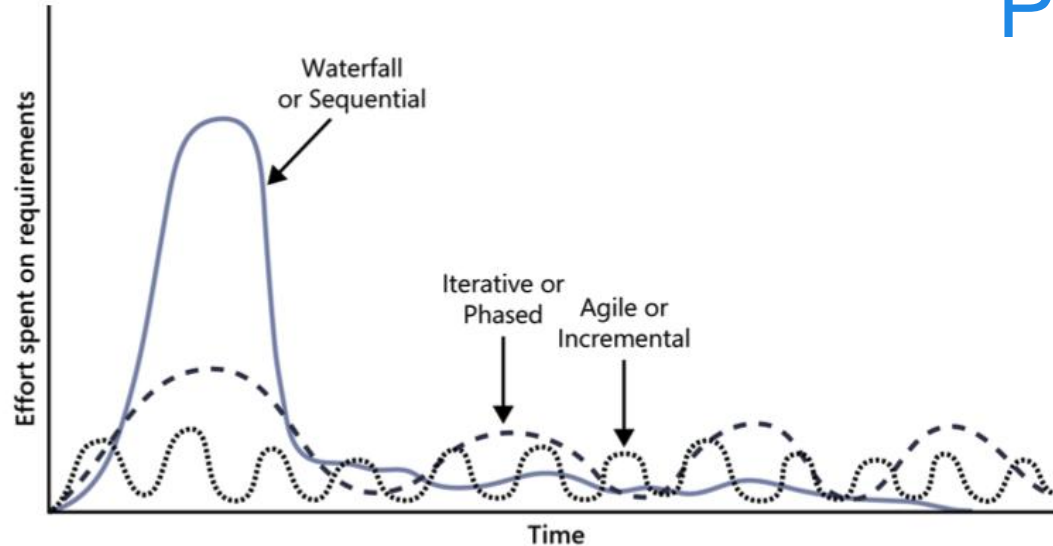


FIGURE 3-3 The distribution of requirements development effort over time varies for projects that follow different development life cycles.

Requirements Engineering

Process II

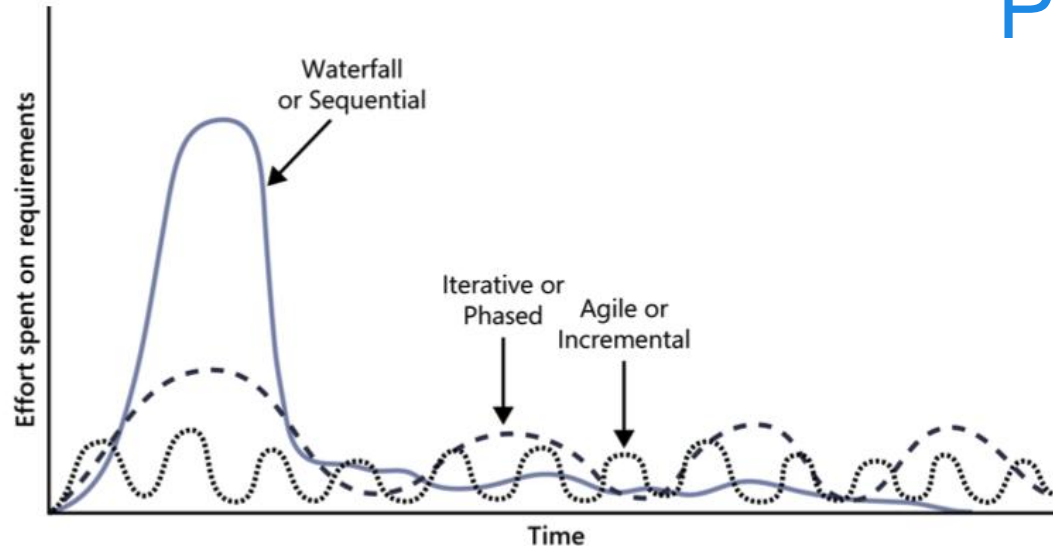


FIGURE 3-3 The distribution of requirements development effort over time varies for projects that follow different development life cycles.

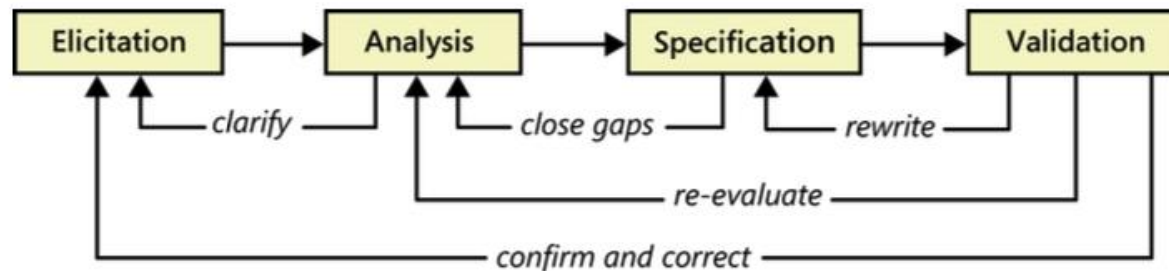


FIGURE 3-1 Requirements development is an iterative process.

Requirements Engineering.

Elicitation

- The **output** of requirements development is a **common understanding of the needs** of **stakeholders**
- When **developers** understand the needs, they can explore alternative solutions

- **Process**

— what really happens — too messy

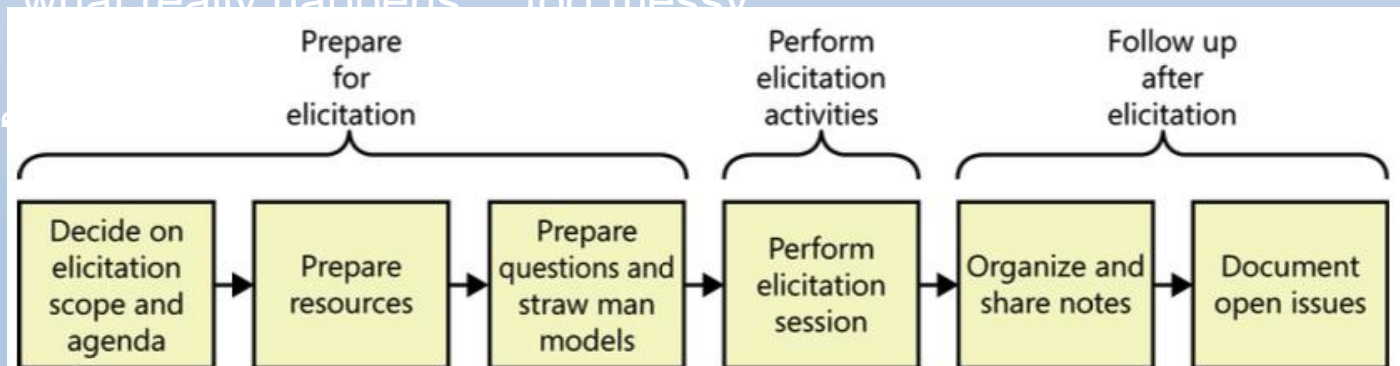


FIGURE 7-2 Activities for a single requirements elicitation session.

Requirements Engineering.

Elicitation

- **Common methods**
 - Interviews
 - Workshops
 - Focus groups
 - Observations
 - Questionnaires
 - System interface analysis
 - User interface analysis
 - Document analysis

Requirements Engineering.

Elicitation using Interviews

- **Interviews** are the most common methodology
- **Guidelines:**
 - Establish a report (e.g., introductions, goals, educate stakeholders)
 - Stay in scope
 - Prepare questions ahead of time
 - Suggest ideas (but not technical solutions)
 - Listen actively and take good notes
 - Organize and share notes

Use in Project!

Requirements Engineering (Elicitation) – Common problems

1. Unclear terminology

- Example: “*a life insurance policy is close*”
- Solution: add a glossary with all “technical” terms.

Use in Project!

Requirements Engineering (Elicitation) – Common problems

2. Unclear terminology

- Vague, untestable requirements / Amalgamation, lack of organization
- Solutions:
 - A test scenario must be included
 - Numbered items
 - Standard formats
 - Traceability

Use in Project!

Requirements Engineering (Elicitation) – Common problems

3. Things that are not required occur

– Solutions:

- Rationale must be included
- Traceable to stakeholder
- Mandatory/desired

Use in Project!

Requirements Engineering (Elicitation) – Common problems

4. Incomplete

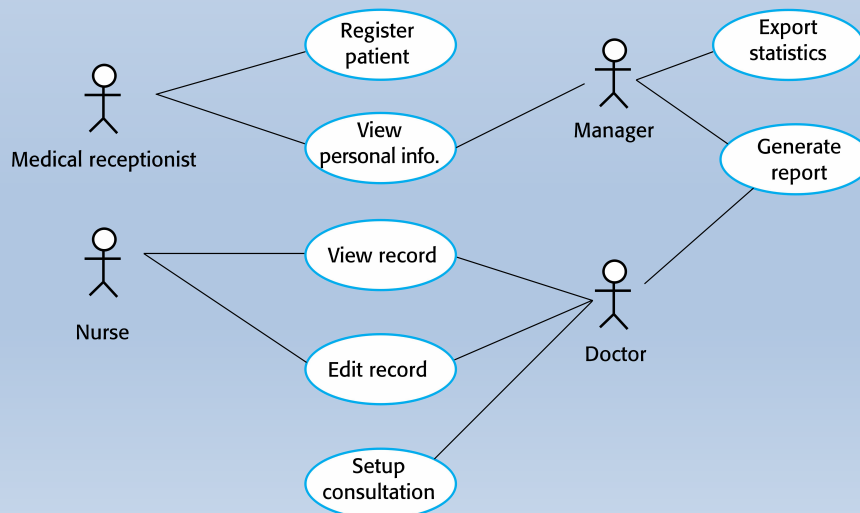
– Solutions:

- Formal model
- Users manual
- Use cases

Use in Project!

Requirements Engineering Specification

- **Use** *natural language + models*
- **Structured as** *use cases*
- In **Agile** are represented by user stories and task cards



Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary id for the generic drug name, lookup the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.

Requirements Engineering Specification

- **Process** to identify use cases
 - Identify the actors first, then
 - Lay out how the domain influences actors, and
 - Define the use cases for activities where actors and systems interact.
- Create a specific scenario to illustrate each process, then
- Generalize into use cases and identify the actors involved
- Ask, *“What tasks must the system perform to complete this process or convert the inputs into outputs?”*
- Those tasks might be use cases.

Requirements Engineering Specification

ID and Name:	UC-4 Request a Chemical		
Created By:	Lori	Date Created:	8/22/13
Primary Actor:	Requester	Secondary Actors:	Buyer, Chemical Stockroom, Training Database
Description:	The Requester specifies the desired chemical to request by entering its name or chemical ID number or by importing its structure from a chemical drawing tool. The system either offers the Requester a container of the chemical from the chemical stockroom or lets the Requester order one from a vendor.		
Trigger:	Requester indicates that he wants to request a chemical.		
Preconditions:	PRE-1. User's identity has been authenticated. PRE-2. User is authorized to request chemicals. PRE-3. Chemical inventory database is online.		
Postconditions:	POST-1. Request is stored in the CTS. POST-2. Request was sent to the Chemical Stockroom or to a Buyer.		
Normal Flow:	4.0 Request a Chemical from the Chemical Stockroom 1. Requester specifies the desired chemical. 2. System lists containers of the desired chemical that are in the chemical stockroom, if any. 3. System gives Requester the option to View Container History for any container. 4. Requester selects a specific container or asks to place a vendor order (see 4.1). 5. Requester enters other information to complete the request. 6. System stores the request and notifies the Chemical Stockroom.		
Alternative Flows:	4.1 Request a Chemical from a Vendor 1. Requester searches vendor catalogs for the chemical (see 4.1.E1). 2. System displays a list of vendors for the chemical with available container sizes, grades, and prices. 3. Requester selects a vendor, container size, grade, and number of containers. 4. Requester enters other information to complete the request. 5. System stores the request and notifies the Buyer.		
Exceptions:	4.1.E1 Chemical Is Not Commercially Available 1. System displays message: No vendors for that chemical. 2. System asks Requester if he wants to request another chemical (3a) or to exit (4a). 3a. Requester asks to request another chemical. 3b. System starts normal flow over. 4a. Requester asks to exit. 4b. System terminates use case.		
Priority:	High		
Frequency of Use:	Approximately 5 times per week by each chemist, 200 times per week by chemical stockroom staff		
Business Rules:	BR-28, BR-31		
Other Information:	The system must be able to import a chemical structure in the standard encoded form from any of the supported chemical drawing packages.		
Assumptions:	Imported chemical structures are assumed to be valid.		

Details! Do not memorize

Requirements Engineering (Specification)

- **Structure of Requirements Document I**

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

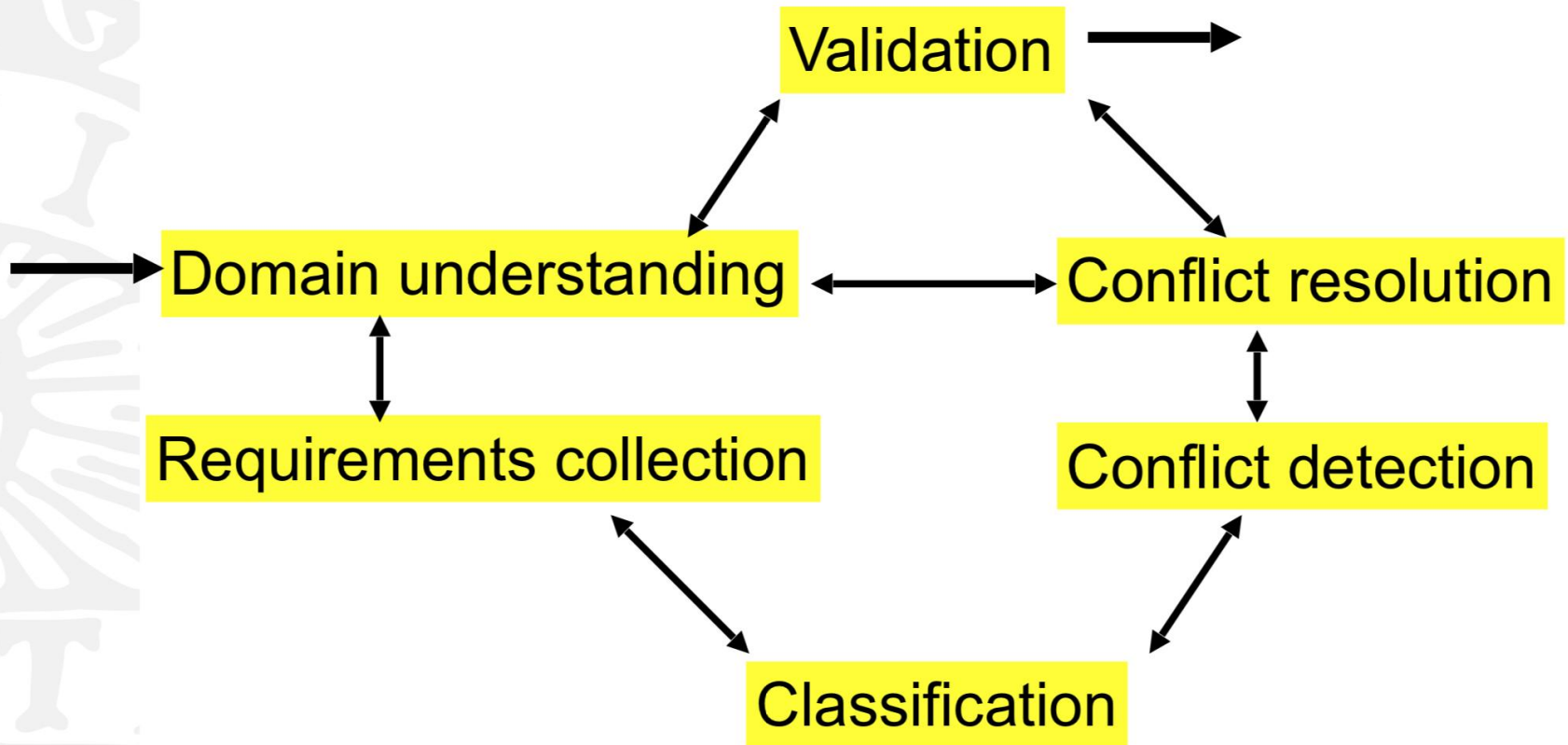
Details! Do not memorize

Requirements Engineering (Specification)

- **Structure of Requirements Document II**

Chapter	Description
S y s t e m r e q u i r e m e n t s specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirements Engineering (Validation)



How do we get the requirements right? (*Validation*)

- **User review of**
 - *Requirements*
 - *Models*
 - *Prototype*
 - *Test cases*
- **Comparison with similar systems**

Requirements Engineering Checklist

Statements Verification

- **Complete** *(has all information necessary for reader)*
- **Correct** *(describes a stakeholder need)*
- **Feasible** *(is this possible given the limitations)*
- **Necessary** *(is this required, rationale)*
- **Prioritized**
- **Unambiguous**
- **Verifiable**

Requirements Engineering

Collection Verification

- **Valid**
 - *Real users*
- **Realistic**
 - *On time / Under budget / Current tech*
- **Complete**
 - *Is all functionality captured? / Are all cases covered?*
- **Consistent**
 - *Internal to requirements*
- **Verifiable**
 - *Can requirements be tested?*

Requirements Engineering.

Exercise

- Example 1. The Background Task Manager shall provide status messages at regular intervals not less than every 60 seconds.
- Problems.
 - What are the status messages?
 - Under what conditions and in what fashion are they provided to the user?
 - If displayed on the screen, how long do they remain visible?
 - Is it okay if they just flash up for half a second?
 - The timing interval is not clear, and the word “every” just muddles the issue.

Requirements Engineering.

Exercise

- Example 1. The Background Task Manager shall provide status messages at regular intervals not less than every 60 seconds.
- Problems.
 - What are the status messages?
 - Under what conditions and in what fashion are they provided to the user?
 - If displayed on the screen, how long do they remain visible?
 - Is it okay if they just flash up for half a second?
 - The timing interval is not clear, and the word “every” just muddles the issue.

Requirements Engineering.

Exercise

- Example 1. The Background Task Manager shall provide status messages at regular intervals not less than every 60 seconds.
- Proposal
 - 1. The Background Task Manager (BTM) shall display status messages in a designated area of the user interface.
 - 1.1. The BTM shall update the messages every 60 plus or minus 5 seconds after background task processing begins.
 - 1.2. The messages shall remain visible continuously during background processing.
 - 1.3. The BTM shall display the percent of the background task that is completed.
 - 1.4. The BTM shall display a “Done” message when the background task is completed.
 - 1.5. The BTM shall display a message if the background task has stalled.

References

	9ed	10ed
Chapters	2.2.1 4 10.3	2.2.1 Software specification, 4 Requirements engineering, 19.3 system procurement
Important Sections	2.2.1, 4-in, 4.1, 4.2, 4.3-in, 4.3.1, 4.5, 4.6, 4.7	2.2.1 Software specification, 4.1 Functional and non-functional requirements, 4.3 Requirements elicitation, 4.3.1 Natural language specification, 4.4 Requirements specification, 4.4.4 Software requirements document, 4.5 Requirements validation, 4.6 Requirements change

Questions

